TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Design and Development of a CRUD Application Integrated with an SQL Database

Submitted By:

Binay Kumar Chand(L.E.082BCT02)

Submitted To:

BCT-III/I

Thapathali Campus

Kathmandu, Nepal

Under the Supervision of

Rajad Shakya

2026

1. ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to my respected teacher, Mr. Anup Shrestha, for his guidance, encouragement, and continuous support during the preparation of this DBMS project report. His suggestions helped me shape this project into a more practical and meaningful academic work.

I am equally thankful to Thapathali Campus, Institute of Engineering, for providing the academic environment, resources, and opportunity to study the practical application of database systems. The course has helped me understand that database concepts are not limited to theory; they directly support the design of useful information systems in the real world.

I would also like to thank my classmates, friends, and family members who motivated me throughout the completion of this report. Their support helped me stay focused and improve the quality of the work. Finally, I am grateful to the authors of textbooks, official documentation, and learning materials related to SQL, DBMS, Python, and web development, which helped me strengthen both the theoretical and practical side of this project.

2. ABSTRACT

This report presents the design and development of a CRUD application integrated with an SQL database, using a Student Management System as the working example. CRUD stands for Create, Read, Update, and Delete, which represent the four fundamental operations performed on data stored in a database. These operations form the operational backbone of most modern information systems such as student portals, hospital systems, banking software, online shopping platforms, and inventory management applications.

The project demonstrates how a software application can connect to a relational database, accept user input through an interface, process that input through application logic, and then store or retrieve data through structured SQL queries. The report discusses the practical role of DBMS in application development, the need for data integrity, the use of validation, and the importance of efficient record handling. It also explains the system architecture, entity design, relational schema, SQL statements, modules, workflows, testing procedures, and implementation logic.

In addition to theoretical explanation, the report emphasizes how the project can be implemented in practice using Python and MySQL. The overall objective is to bridge the gap between classroom understanding of databases and the actual process of developing a database-driven application. The completed system shows how a structured SQL database can improve accuracy, consistency, accessibility, and maintainability of data compared with manual record-keeping systems.

3. TABLE OF CONTENTS

1. Introduction

2. Objectives

3. Problem Statement

4. Scope of the Project

5. Technologies Used

6. Understanding CRUD Operations

7. System Architecture

8. Database Design

9. ER Diagram Explanation

10. Relational Schema

11. SQL Queries Used

12. Application Modules

13. Working of the System

14. Input and Output Design

15. Validation and Error Handling

16. Testing of the Application

17. Advantages of the System

18. Limitations

19. Future Enhancements

20. Conclusion

21. References

Appendix A - Sample SQL Script

Appendix B - Python + MySQL Implementation Overview

Appendix C - Viva Questions and Answers

4. INTRODUCTION

Database Management Systems (DBMS) are software systems designed to define, create, maintain, and control access to databases. In an academic sense, DBMS provides the theoretical foundation for storing data in a structured form; in a practical sense, it makes it possible to build applications that can handle thousands of records with speed, accuracy, and consistency. Almost every modern digital service depends on a database at some stage, whether it is used for customer registration, product details, account information, attendance, exam marks, online orders, or health records.

A CRUD application is the simplest and most important practical demonstration of database usage. CRUD stands for Create, Read, Update, and Delete. Together, these four operations represent the minimum set of actions required to manage records in a persistent storage system. When these operations are connected with a relational database such as MySQL, the application becomes a powerful tool for record management.

In this project, the concept is implemented using a Student Management System. The user can add new student records, view all records, modify existing information, and remove unnecessary data. Although the example is academic in nature, the same design pattern can be used in real systems for employee management, inventory control, hospital administration, booking platforms, and customer relationship management. Therefore, the project is not only useful for course completion but also for building foundational software development skills.

5. OBJECTIVES

The purpose of this project is not merely to create a small software interface, but to understand how a database-oriented system is planned, modeled, built, and tested from beginning to end. Through this report, the project aims to translate DBMS theory into a complete working model that demonstrates how structured data is managed in practice.

- To understand the practical significance of CRUD operations in application development.
- To design a relational database schema capable of storing student records in a structured and normalized form.
- To connect a software application with MySQL and perform database transactions through SQL statements.
- To implement record insertion, viewing, editing, and deletion in a user-friendly way.
- To apply validation rules so that the system minimizes invalid or duplicate data entry.
- To observe how frontend, backend, and database components communicate in a three-tier architecture.
- To gain hands-on experience that can be extended later into larger DBMS projects.

6. PROBLEM STATEMENT

In many small institutions and offices, records are still maintained manually using paper files, notebooks, or simple spreadsheet lists. Although such methods may work for a very small amount of data, they quickly become inefficient when the number of records increases. Searching for a particular record becomes slow, updating information is inconvenient, duplicate entries may appear, and there is a high chance of human error.

A student record system is one example where accuracy is very important. If the system stores incorrect or duplicate roll numbers, outdated phone numbers, or missing department details, it affects communication, reporting, and administrative work. Manual systems also lack proper validation and often make it difficult to keep records consistent over time.

To solve these issues, this project proposes a CRUD-based database application integrated with an SQL database. The proposed system stores records in a structured relational format, supports easy insertion and retrieval, allows controlled updating, and enables safe deletion of data. The problem, therefore, is addressed by replacing informal data storage with a proper database-driven application.

7. SCOPE OF THE PROJECT

The scope of this project includes requirement identification, database design, implementation of a relational table, and development of a simple application interface connected to MySQL. The project focuses on the end-to-end management of student records and demonstrates the standard workflow of a database application.

Within this scope, the system covers form-based data entry, tabular display of records, update and delete actions, and validation at both the application and database level. The project also includes testing of common scenarios such as valid insertion, duplicate input, empty fields, and search functionality.

However, the project is intentionally limited to a beginner-to-intermediate academic level. It does not include advanced distributed architecture, full production security, extensive analytics, or enterprise deployment. Even so, the system provides a strong and practical foundation for understanding how real-world DBMS applications are structured.

8. TECHNOLOGIES USED

A CRUD application can be developed using different technology stacks depending on the skill level of the developer, the operating environment, and project requirements. For this report, Python with Flask is used as the backend framework, HTML and CSS are used to build the interface, and MySQL serves as the relational database. This stack is suitable for academic work because it is lightweight, easy to understand, and highly practical for demonstrating the interaction between software and a database.

8.1 Frontend Technologies

- HTML (HyperText Markup Language): Used to define the structure of web pages such as forms, tables, labels, buttons, and navigation links.

- CSS (Cascading Style Sheets): Used to improve visual presentation, spacing, alignment, colors, and readability of the interface.

- Optional JavaScript: Can be added for confirmation prompts, simple client-side validation, and dynamic behavior, although the application can also work without heavy JavaScript.

8.2 Backend Technologies

- Python: A high-level, beginner-friendly programming language widely used in web applications, automation, and data processing.

- Flask: A lightweight Python web framework used to define routes, receive form data, connect to the database, and return rendered pages.

- mysql-connector-python or Flask-MySQLdb: Libraries that establish communication between Python code and the MySQL server.

8.3 Database Technology

- MySQL: An open-source relational database management system that stores records in tables and supports standard SQL operations.

- SQL: The structured query language used to define the schema and perform insert, select, update, and delete commands.

8.4 Why Python + MySQL is Suitable for This Project

Python + MySQL is appropriate for a college-level DBMS project because it demonstrates all essential database concepts while remaining simple to install, execute, and explain in viva examinations. The code is readable, the SQL operations are explicit, and the architecture remains clear enough for academic presentation.

9. UNDERSTANDING CRUD OPERATIONS

CRUD operations represent the lifecycle of data inside a management system. Every record begins with creation, is later viewed and used, may be updated when information changes, and can eventually be deleted if it is no longer needed. A well-designed CRUD system ensures that each of these operations is performed safely, accurately, and efficiently.

9.1 Create

The Create operation inserts a new row into the database. In the Student Management System, this takes place when the user fills a form with student name, roll number, email, department, phone number, and address. Before the data is inserted, the application should verify that required fields are present and that unique values such as roll number do not already exist.

```
INSERT INTO students (name, roll_no, email, department, phone, address)
VALUES ('Ram Sharma', '101', 'ram@gmail.com', 'Computer Science',
'9801234567', 'Kathmandu');
```

9.2 Read

The Read operation retrieves data already stored in the database. This may involve displaying all records in a table, searching for a particular roll number, or viewing the complete details of one student. In practice, Read operations are the most frequent because users often need to consult existing information.

```
SELECT * FROM students;
```

9.3 Update

The Update operation changes an existing row. For example, if a student changes address or phone number, it is not correct to create a new record; the existing record should be modified. The update feature ensures that the same primary identity remains intact while selected fields change.

```
UPDATE students
SET phone = '9811111111', address = 'Lalitpur'
WHERE student_id = 1;
```

9.4 Delete

The Delete operation removes a record from the database. This should usually be performed carefully because it permanently erases information unless backup or soft-delete logic is used. In the student system, a deletion may be necessary if a duplicate entry was created or if test data needs to be removed.

```
DELETE FROM students
WHERE student_id = 1;
```

10. SYSTEM ARCHITECTURE

The project follows a three-tier architecture. This separation is important because it improves clarity, maintainability, and scalability. Each layer has a defined responsibility, and together they create a clear path from user action to stored data.

10.1 Presentation Layer

This is the visible interface used by the end user. It contains input forms, buttons, table views, and feedback messages. In this project, the presentation layer is implemented using HTML templates styled with CSS. Good interface design matters because even a technically correct application becomes difficult to use if the inputs are unclear or the data table is poorly organized.

10.2 Application Layer

This layer contains the business logic of the system. It receives requests from the user interface, validates form inputs, constructs or executes SQL queries, handles exceptions, and decides what response should be shown. In the Python implementation, Flask route functions represent this layer.

10.3 Database Layer

This layer is responsible for permanent data storage. It includes the MySQL server, the student_management database, and the students table. It enforces constraints such as primary keys, unique values, and non-null fields.

10.4 Flow of Data Through the System

- The user opens a page and enters data through a form.
- The application receives the submitted request.
- Validation rules check whether the input is complete and acceptable.
- A SQL query is executed through the database connector.

- MySQL processes the query and returns a result.

- The application prepares a message or updated page and displays it to the user.

11. DATABASE DESIGN

Database design is a critical stage because the quality of the application depends heavily on how well the data model reflects the real-world system. Even a simple CRUD application should have clearly identified entities, attributes, keys, and constraints. In this project, the primary entity is Student.

11.1 Entity Identification

An entity is any real-world object or concept about which data needs to be stored. Here, the system stores information about students; therefore Student is the central entity.

11.2 Attributes

The attributes chosen for the Student entity represent core identification and contact details required by a basic academic information system.

Attribute	Purpose	Reason for Inclusion
student_id	Primary identifier	Ensures each row can be uniquely identified and updated or deleted reliably.
name	Student's full name	Used for display, identification, and search.
roll_no	Institutional roll number	A unique academic identifier that should not repeat.
email	Contact email	Useful for communication and additional uniqueness.
department	Academic department/program	Helps categorize students by field or section.
phone	Contact number	Stores an immediate contact detail.
address	Residential/permanent address	Useful for administrative records.

11.3 Table Design and Constraints

The students table is designed with one surrogate primary key and several descriptive attributes. Constraints ensure that the database itself participates in data integrity. For example, name and department should not be left blank, while roll number should remain unique for each student.

Field Name	Data Type	Constraint	Description
student_id	INT	PRIMARY KEY, AUTO_INCREMENT	Numeric identifier automatically generated by the database.
name	VARCHAR(100)	NOT NULL	Stores the student's full name.
roll_no	VARCHAR(20)	UNIQUE, NOT NULL	Ensures no two students share the same roll number.
email	VARCHAR(100)	UNIQUE	Stores email address; uniqueness avoids duplicates.
department	VARCHAR(50)	NOT NULL	Stores department or program name.
phone	VARCHAR(15)	Optional	Stores phone number.
address	VARCHAR(255)	Optional	Stores address information.

12. ER DIAGRAM EXPLANATION

An Entity-Relationship (ER) diagram is a conceptual representation of the data model. It helps visualize entities, their attributes, and the relationships among them. Since this project is intentionally simple, the diagram contains a single main entity named Student.

12.1 Conceptual Representation

The Student entity has one primary key, student_id, and several descriptive attributes: name, roll_no, email, department, phone, and address. Because the project uses only one entity for the basic version, no inter-table relationships are required. This makes the ER model easy to understand for beginners.

12.2 Textual ER Representation

```
Student(student_id, name, roll_no, email, department, phone, address)
```

12.3 Possible Extended ER Model

In an advanced version of the system, additional entities could be introduced such as Department, Course, Enrollment, Attendance, or Result. In that case, one department could be related to many students, and one student could be related to many courses through an enrollment table. Mentioning this extension shows that the current design is scalable.

13. RELATIONAL SCHEMA

The relational schema is the formal table definition derived from the ER model. It identifies the relation name, field names, data types, keys, and constraints. The schema for this project is shown below.

```
STUDENTS(
    student_id        INT        PRIMARY        KEY        AUTO_INCREMENT,
    name               VARCHAR(100)               NOT               NULL,
    roll_no         VARCHAR(20)         UNIQUE         NOT         NULL,
    email                      VARCHAR(100)                      UNIQUE,
    department             VARCHAR(50)             NOT             NULL,
    phone                                                    VARCHAR(15),
    address                                                  VARCHAR(255)
)
```

This schema is already sufficient for normal CRUD operations. It avoids unnecessary complexity and keeps the design easy to explain in an academic setting.

14. SQL QUERIES USED

SQL statements are the bridge between the application and the database. They define structure, insert data, retrieve data, modify existing values, and remove rows when required.

14.1 Create Database

Creates a new database to store all project tables and records.

```
CREATE DATABASE student_management;
```

14.2 Select Database

Makes the student_management database active for subsequent operations.

```
USE student_management;
```

14.3 Create Table

Creates the students table with all required fields and constraints.

```
CREATE                  TABLE                  students                  (
    student_id        INT        AUTO_INCREMENT        PRIMARY        KEY,
    name               VARCHAR(100)               NOT               NULL,
    roll_no         VARCHAR(20)         UNIQUE         NOT         NULL,
    email                      VARCHAR(100)                      UNIQUE,
    department             VARCHAR(50)             NOT             NULL,
    phone                                                   VARCHAR(15),
    address                                                 VARCHAR(255)
);
```

14.4 Insert Record

Adds a new student record to the table.

```
INSERT INTO students (name, roll_no, email, department, phone, address)
VALUES  ('Sita  Rai',  '102',  'sita@gmail.com',  'BCA',  '9800000000',
```

```
'Kathmandu');
```

14.5 View All Records

Displays all existing student records.

```
SELECT * FROM students;
```

14.6 Search Specific Record

Retrieves records matching a given roll number.

```
SELECT * FROM students
WHERE roll_no = '102';
```

14.7 Update Record

Changes one or more field values in an existing row.

```
UPDATE students
SET phone = '9811111111', address = 'Lalitpur'
WHERE student_id = 1;
```

14.8 Delete Record

Removes a selected row from the table.

```
DELETE FROM students
WHERE student_id = 1;
```

15. APPLICATION MODULES

Dividing the application into modules makes the system easier to understand, maintain, test, and expand. Each module represents a logical unit of work in the application.

15.1 Add Student Module

This module provides a form through which the user enters student details. After submission, the application validates the input and then runs the INSERT query.

Typical fields include name, roll number, email, department, phone number, and address.

The module should return a success message when the record is saved correctly or a clear error message when validation fails.

15.2 View Student Module

This module fetches all student records from the database using a SELECT query and displays them in a table.

It is the main monitoring module because the user can quickly inspect all available records.

In many systems, this page also contains links or buttons for editing and deleting records.

15.3 Update Student Module

This module first identifies a specific record, usually by student_id. It then loads the existing values into a form.

After the user edits the values and submits the form, the application applies validation and executes an UPDATE query.

The module is important because it allows correction of outdated or incorrect information without creating duplicate entries.

15.4 Delete Student Module

This module removes a student record from the table.

A confirmation message is usually shown before deletion so that accidental record loss can be avoided.

In a more advanced system, a soft-delete strategy may be used instead of permanent deletion.

15.5 Search Module

This module enables the user to search records by roll number, student ID, or name.

It improves usability because it is inefficient to scan a large table manually.

The search module may use the WHERE clause, LIKE operator, or exact match conditions.

16. WORKING OF THE SYSTEM

The working of the system can be understood as a sequence of actions performed by the user and processed by the application. The steps below describe the operational flow in a simple but realistic way.

16.1 Step 1: Application Startup

The user runs the Flask application and opens the local web address in a browser. The homepage presents navigation options such as Add Student and View Students.

16.2 Step 2: Record Creation

The user fills in the form with valid details and clicks the submit button. The server receives the form data, validates it, and inserts it into MySQL. A confirmation message is shown if the operation succeeds.

16.3 Step 3: Record Display

The View Students page executes a SELECT query. The returned rows are displayed in a structured table so the user can inspect all stored data.

16.4 Step 4: Record Update

The user clicks an Edit option associated with a selected row. The existing record is loaded into a form, modified values are submitted, and the UPDATE statement saves the changes.

16.5 Step 5: Record Deletion

The user clicks Delete for a selected record. After confirmation, the system executes a DELETE statement and refreshes the listing.

16.6 Step 6: Continuous Data Management

The same cycle can continue repeatedly as new records are added and old records are maintained. This reflects how CRUD applications are used in real administrative environments.

17. INPUT AND OUTPUT DESIGN

17.1 Input Design

Input design determines how users provide data to the system. Good input design reduces mistakes, improves clarity, and ensures that the database receives information in the expected format. For this project, a structured form is used.

- Student Name - text field; should not be empty.
- Roll Number - text field; should remain unique.
- Email - text field; should follow email format.
- Department - text field or dropdown depending on implementation.
- Phone - text field restricted to numeric values if possible.
- Address - text area for longer text.

Labels should be clear and placed close to their corresponding input fields. Required fields should be marked visibly. The form should also provide feedback messages when a problem occurs.

17.2 Output Design

Output design controls how retrieved data is shown to the user. The application uses a table-based display because it is ideal for representing multiple records at once. The columns correspond to the database fields, and each row represents one student.

ID	Name	Roll No.	Email	Department	Phone	Address
1	Sita Rai	102	sita@gmail.com	BCA	9800000000	Kathmandu
2	Ram Sharma	101	ram@gmail.com	Computer Science	9801234567	Pokhara

Alongside each row, action controls such as Edit and Delete may be provided. This design allows the user to perform record maintenance directly from the output page.

18. VALIDATION AND ERROR HANDLING

Validation and error handling are essential because a system is only reliable when it can prevent incorrect data from entering the database and respond gracefully when problems occur. Validation should be applied in two complementary layers: application-level validation and database-level validation.

18.1 Application-Level Validation

- Name should not be left blank.
- Roll number should not be empty and should follow a defined pattern if required.
- Email should contain a valid format such as text@domain.com.
- Department should not be empty.
- Phone number should contain only digits or accepted symbols.
- Whitespace-only inputs should be treated as invalid.

18.2 Database-Level Validation

Even if the application performs checks, the database must also enforce integrity rules. This is important because data may later come from another application or import process.

- PRIMARY KEY guarantees uniqueness of the student_id field.
- UNIQUE constraints stop duplicate roll numbers or duplicate emails.
- NOT NULL constraints ensure that essential fields always have values.

18.3 Error Handling Examples

- Duplicate entry error when a roll number already exists.

- Database connection error when the MySQL server is not running or credentials are incorrect.

- Record not found when an update or delete request targets a non-existing ID.

- Form submission error when required values are missing.

- Unexpected server exceptions, which should be caught and shown as understandable messages instead of raw tracebacks in a production system.

19. TESTING OF THE APPLICATION

Testing is necessary to verify that each module behaves as expected. It also demonstrates that the project has been evaluated methodically rather than assumed to work correctly. Functional testing is the most relevant for a CRUD project.

Test Case	Input / Action	Expected Result
Insert valid record	Enter all valid student details and submit	Record is stored successfully and appears in the list.
Insert duplicate roll number	Use an existing roll number	System rejects the input and shows an error message.
View records	Open the view page	All available student records are displayed.
Update existing record	Edit phone number or address of an existing student	Changes are saved and shown after refresh.
Delete record	Delete one selected student	Record is removed from the list and database.
Search non-existing record	Search for an invalid ID or roll number	System returns 'Record not found' or empty result.
Blank field validation	Submit form with missing name or department	System blocks insertion and requests correction.

19.1 Importance of Testing

Testing confirms not only correctness but also usability. A CRUD system that technically runs but fails under basic input variation cannot be considered complete. Therefore, testing should be mentioned confidently during report submission and viva.

20. ADVANTAGES OF THE SYSTEM

- Provides organized and structured storage of student information.
- Greatly reduces the time required to search, update, and manage records.
- Minimizes data duplication through keys and unique constraints.
- Improves accuracy through validation and controlled updates.
- Supports easy expansion into a larger management system.
- Acts as a practical demonstration of relational database concepts.
- Can be deployed locally in labs, classrooms, or small office environments.
- Builds a strong academic bridge between DBMS theory and software development practice.

21. LIMITATIONS

As a college-level project, the system is intentionally simple. This simplicity is useful for learning, but it also creates limitations that should be acknowledged honestly.

- The system usually supports only one primary entity in its beginner version.
- Security measures such as authentication, encryption, and session control may be minimal.
- The project may not handle large-scale concurrent access efficiently without further optimization.
- Backup, recovery, and transaction management are not explored deeply in the basic implementation.
- Advanced reporting, charting, and analytics are outside the current scope.
- Permanent deletion may not be ideal for real-world institutional use without an archive mechanism.

22. FUTURE ENHANCEMENTS

Although the current version is intentionally straightforward, it can be improved in many meaningful ways. Future enhancement ideas show the scalability of the design and the student's understanding of what production systems require.

- Add a login system with username and password for administrators.
- Introduce role-based access control for admin, staff, and viewer roles.
- Create separate tables for departments, courses, and enrollment history.
- Add pagination and advanced filtering for large data sets.
- Export records to PDF or Excel for administrative reporting.
- Maintain audit logs to track who changed what and when.
- Add dashboards showing summary counts and charts.
- Improve the interface for mobile responsiveness.
- Use prepared statements and stronger security practices to reduce SQL injection risk.
- Deploy the application on a cloud or campus server for multi-user access.

23. CONCLUSION

This project demonstrates how DBMS concepts are transformed into a practical software solution through a CRUD application integrated with an SQL database. By designing a student table, defining SQL queries, and creating a simple interface connected through Python, the project makes the role of databases clear and tangible.

The implementation confirms that even a relatively small application requires careful attention to schema design, data integrity, validation, and usability. The four CRUD operations form the functional core of almost every record management system, and understanding them is essential for any student of database systems.

In conclusion, the Student Management CRUD Application is an effective academic project because it is simple enough to understand, practical enough to demonstrate real database interaction, and flexible enough to be expanded into larger systems in the future. It therefore serves as a strong foundation for more advanced DBMS and software engineering work.

24. REFERENCES

1. Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems.

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts.

3. MySQL Reference Manual and official documentation.

4. Flask Documentation and Python package documentation for MySQL connectivity.

5. Class notes, lab sheets, and DBMS course materials provided during the semester.

25. APPENDIX A - SAMPLE SQL SCRIPT

```
CREATE                    DATABASE                    student_management;
USE                                                   student_management;

CREATE                  TABLE                  students                  (
    student_id        INT        AUTO_INCREMENT        PRIMARY        KEY,
    name               VARCHAR(100)               NOT               NULL,
    roll_no         VARCHAR(20)         UNIQUE         NOT         NULL,
    email                      VARCHAR(100)                      UNIQUE,
    department             VARCHAR(50)             NOT             NULL,
    phone                                                   VARCHAR(15),
    address                                                 VARCHAR(255)
);

INSERT INTO students (name, roll_no, email, department, phone, address)
VALUES
('Ram   Sharma',   '101',   'ram@gmail.com',   'BCT',   '9801234567',
'Kathmandu'),
('Sita Rai', '102', 'sita@gmail.com', 'BCA', '9800000000', 'Lalitpur');
```

26. APPENDIX B - PYTHON + MYSQL IMPLEMENTATION OVERVIEW

The Python + MySQL version of the project typically contains an app.py file, a templates directory for HTML files, a static directory for CSS, and a db.sql script used to initialize the database. Flask route functions are mapped to application actions such as displaying the homepage, inserting a student, editing a student, and deleting a student.

- Route / or /students - displays all students using a SELECT query.
- Route /add - renders and processes the insertion form.
- Route /edit/<id> - loads one record for editing and saves updates.
- Route /delete/<id> - deletes one selected record after confirmation.

The backend opens a database connection using host, user, password, and database parameters. Queries are executed through a cursor object. After insert, update, or delete operations, commit() is called to save changes.

27. APPENDIX C - VIVA QUESTIONS AND ANSWERS

27.1 What does CRUD stand for?

CRUD stands for Create, Read, Update, and Delete.

27.2 Why is MySQL used in this project?

MySQL is a relational database system that stores the records in tables and allows SQL-based manipulation of data.

27.3 What is the role of Flask in the project?

Flask acts as the backend framework that receives requests, processes logic, connects to the database, and returns pages.

27.4 Why is student_id used as a primary key?

It uniquely identifies each record and allows reliable update and deletion of rows.

27.5 What is the difference between DELETE and DROP?

DELETE removes rows from a table, while DROP removes the table structure itself.

27.6 Why do we use validation?

Validation prevents incorrect, incomplete, or duplicate data from being stored in the database.

27.7 Can this project be expanded?

Yes. It can be extended with login, multiple tables, reporting, search filters, export options, and deployment.

THANK YOU